CkaDinTi
Arron's Project Tasks

Owner: teacher desktop, lesson ingestion, AI generation, quiz review, and teacher reports

Build the trusted teacher workflow from lesson import through approved quiz and game setup.

Current database decision

The research report originally recommended SQLite. The team has changed the persistence choice to PostgreSQL. Use PostgreSQL + Drizzle throughout the teacher content pipeline and coordinate shared schema changes with Edin. This task assignment supersedes the report's SQLite recommendation.

Milestone A - teacher foundation

- Scaffold the pnpm workspace and Electron Forge teacher app.
- Create secure Electron main/preload boundaries; keep secrets and filesystem access out of the renderer.
- Implement PostgreSQL/Drizzle repositories and versioned SQL migrations for lessons, lesson blocks, quizzes, questions, options, evidence, and AI jobs.
- Add shared Zod contracts for lesson import, quiz generation, review state, and teacher IPC.
- Connect through DATABASE_URL; keep credentials in trusted main-process configuration and out of renderer bundles.

Milestone B - content pipeline

- Implement TXT, Markdown, text-based PDF, and DOCX ingestion with normalized text and source evidence ranges.
- Add content hashing, deduplication, validation errors, and fixture-based parser tests.
- Implement OpenAI Responses API generation with strict structured output, prompt versioning, evidence, explanation, difficulty, topic, and confidence label.
- Keep teacher approval mandatory; generated questions remain drafts until explicitly approved.

Milestone C - teacher UX

- Build Dashboard, Lessons, Quizzes, Review, Reports, and Settings screens.
- Implement edit, delete, reorder, duplicate, add manually, regenerate question/distractors, approve/reject, and approve-all-reviewed actions.
- Add source-evidence view-in-lesson behavior; editing an approved quiz returns it to review status.
- Build teacher host/lobby screens against Edin's shared contracts without implementing the game server.

Acceptance criteria and tests

- A lesson can be imported, normalized, generated into schema-valid questions, reviewed, edited, and approved locally.
- Approved quiz data can be exported through shared contracts as a stable game-start snapshot.
- No OpenAI key or unrevealed correct answer appears in renderer bundles or student payloads.
- Test parsers, schema validation, review transitions, PostgreSQL migrations/repositories, and persistence.
- Add teacher UI tests for import, review actions, approval, and evidence navigation; run typecheck, tests, and Electron packaging smoke check.

Ownership boundaries

- Arron owns lesson and quiz tables. Edin owns game, session, result, and runtime analytics tables; jointly review migrations that touch both.
- Do not implement Fastify/Socket.IO server logic, authoritative scoring, or student gameplay.
- Publish approved quiz snapshots through packages/contracts; consume Edin's game events for teacher reports without recalculating scores.
- Graphify is optional and must stay behind a teacher-report visualization adapter; it does not own scoring or persistence.
- Use RTK CLI wrappers for repository inspection and test output. Keep commits focused and prefix them with arron:.

References: Graphify - https://github.com/Graphify-Labs/graphify
RTK - https://github.com/rtk-ai/rtk
Repository - https://github.com/CkaDinTi/CkaDinTi